PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS

LUCAS FERNANDES NASCIMENTO

SMARTMARKET: APLICATIVO MÓVEL DE LISTA DE COMPRAS COM REACT NATIVE E FIREBASE

Contagem
2026

LUCAS FERNANDES NASCIMENTO

SMARTMARKET: APLICATIVO MÓVEL DE LISTA DE COMPRAS COM REACT NATIVE E FIREBASE

Relatório apresentado à disciplina de Desenvolvimento de Aplicações Web como atividade avaliativa, descrevendo o desenvolvimento do aplicativo SmartMarket.

Contagem
2026

RESUMO

Este trabalho descreve o desenvolvimento do SmartMarket, um aplicativo móvel de lista de compras de supermercado construído com React Native, por meio da plataforma Expo, e com os serviços Firebase Authentication e Cloud Firestore. O usuário cria uma conta com e-mail e senha, recupera o acesso por e-mail quando esquece a senha e mantém listas de compras com produtos, quantidades e preços, cujo total é recalculado a cada alteração. O relatório apresenta a configuração do Firebase, a modelagem dos dados, as regras de segurança do banco, a organização do código e as telas resultantes, além de indicar o endereço do repositório com o código-fonte e os passos para executá-lo.

Palavras-chave: React Native. Firebase. Cloud Firestore. Aplicativo móvel. Lista de compras.

SUMÁRIO

1 INTRODUÇÃO 5

2 REQUISITOS DA ATIVIDADE 5

3 TECNOLOGIAS UTILIZADAS 6

4 DESENVOLVIMENTO 6

4.1 Criação do projeto e configuração do Firebase 6

4.2 Autenticação 7

4.3 Modelagem dos dados 7

4.4 Regras de segurança 8

4.5 Cálculo dos totais 8

4.6 Interface 9

5 FUNCIONAMENTO DO APLICATIVO 9

6 EXECUÇÃO DO PROJETO 11

7 CONSIDERAÇÕES FINAIS 12

REFERÊNCIAS 13

1 INTRODUÇÃO

A atividade proposta na disciplina de Desenvolvimento de Aplicações Web consistiu em construir um aplicativo móvel de lista de supermercado, chamado SmartMarket, usando React Native e o Firebase Firestore como banco de dados em nuvem. O aplicativo precisava exigir login por e-mail e senha, permitir a recuperação da senha por e-mail e deixar o usuário registrar os produtos que pretende comprar, com os respectivos valores, mostrando o total a pagar sem que ele precise fazer a conta.

Este documento explica como o trabalho foi feito. O código-fonte completo está publicado em um repositório no GitHub, no endereço https://github.com/lucsfn/dam-puc-smartmarket, e é a partir dele que se executa o projeto, conforme a seção 6.

O texto está organizado da seguinte forma: a seção 2 resume o que foi pedido; a seção 3 apresenta as tecnologias escolhidas; a seção 4 descreve as etapas do desenvolvimento, da criação do projeto no Firebase até o cálculo dos totais; a seção 5 mostra o aplicativo funcionando; a seção 6 explica como rodá-lo; e a seção 7 traz as considerações finais.

2 REQUISITOS DA ATIVIDADE

O enunciado reúne os requisitos em quatro grupos. O primeiro é a autenticação: o acesso ao aplicativo se faz por e-mail e senha, com o Firebase Authentication, e o usuário que esquecer a senha deve poder pedir a redefinição por e-mail.

O segundo grupo trata do gerenciamento da lista. O usuário pode criar, editar e excluir produtos, e cada produto traz os campos descritos na Tabela 1. As listas ficam armazenadas no Cloud Firestore.

Tabela 1 – Campos de cada produto exigidos no enunciado

Campo	Tipo	Observação
nomeProduto	string	Obrigatório.
quantidade	number	Obrigatório.
precoUnitario	number	Obrigatório.
dataAdicao	timestamp	Gerado automaticamente.

Fonte: elaborado pelo autor (2026).

O terceiro grupo é o cálculo automático. O aplicativo deve obter o valor total de cada produto, que é a quantidade multiplicada pelo preço unitário, e o valor total da lista, que é a soma dos totais dos produtos. Esse total precisa aparecer em tempo real, conforme o usuário adiciona ou altera itens.

Por fim, a interface deve ter uma tela de login, uma tela de recuperação de senha e uma tela com os produtos da lista selecionada, exibindo o total geral da lista de forma dinâmica.

3 TECNOLOGIAS UTILIZADAS

O aplicativo foi escrito em TypeScript com React Native, utilizando o Expo como plataforma de desenvolvimento (EXPO, [s. d.]). O Expo dispensa a configuração dos projetos nativos de Android e iOS e permite abrir o aplicativo diretamente no celular por meio do aplicativo Expo Go, o que acelera os testes. A versão do SDK usada foi a 58, com React 19 e React Native 0.88.

A navegação entre as telas fica por conta do React Navigation, com a pilha nativa de telas (REACT NAVIGATION, [s. d.]). Para os serviços de nuvem foi utilizado o SDK JavaScript do Firebase, na versão 13, que fornece tanto o Authentication quanto o Cloud Firestore (FIREBASE, [s. d.]a; FIREBASE, [s. d.]b). A escolha pelo SDK JavaScript, e não pelas bibliotecas nativas do Firebase, evita a necessidade de gerar um build de desenvolvimento e de baixar arquivos de configuração como o google-services.json, de modo que o aplicativo funciona no Expo Go e também no navegador.

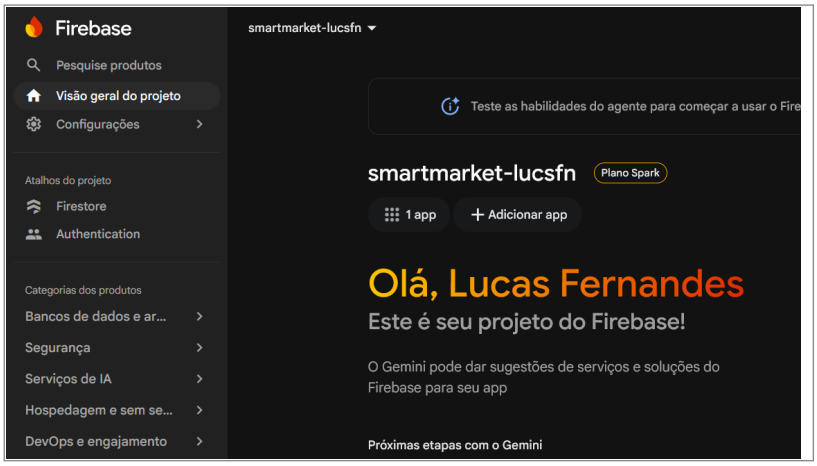
Para manter o usuário conectado depois que o aplicativo é fechado, a sessão do Firebase é gravada no armazenamento do aparelho com o pacote AsyncStorage.

4 DESENVOLVIMENTO

4.1 Criação do projeto e configuração do Firebase

O primeiro passo foi criar o projeto no Firebase Console, chamado smartmarket-lucsfn e mantido no plano gratuito Spark (Figura 1). Dentro dele foi registrado um aplicativo do tipo Web, cujas credenciais (chave de API, identificador do projeto, identificador do aplicativo e demais valores) são exibidas na página de configurações. Em seguida, foi ativado o provedor de e-mail e senha na seção Authentication e foi criado o banco de dados no Cloud Firestore.

Figura 1 – Painel do projeto smartmarket-lucsfn no Firebase Console



Fonte: elaborado pelo autor (2026).

As credenciais não ficam escritas no código. O arquivo src/config/firebase.ts as lê de variáveis de ambiente com o prefixo EXPO_PUBLIC_, prefixo que o Expo exige para incluir a variável no código do aplicativo. Os valores ficam em um arquivo .env, que está listado no .gitignore e por isso não vai para o repositório. No lugar dele, o repositório traz o arquivo .env.example, que lista as seis variáveis necessárias e serve de modelo para quem for executar o projeto.

4.2 Autenticação

A autenticação foi concentrada em um contexto do React, definido em src/context/AuthContext.tsx. Ele oferece quatro operações, todas implementadas com funções do SDK do Firebase: entrar, cadastrar, recuperar senha e sair. O estado do usuário é obtido com a função onAuthStateChanged, que avisa o aplicativo sempre que alguém entra ou sai, inclusive quando a sessão anterior é restaurada ao abrir o aplicativo.

Esse estado também define quais telas existem. No arquivo App.tsx, quando não há usuário logado, a pilha de navegação contém apenas as telas de login e de recuperação de senha. Quando há, contém as telas de listas e de produtos. Dessa forma não é possível chegar às listas sem estar autenticado, e o aplicativo troca de telas sozinho assim que o login é concluído.

Os erros devolvidos pelo Firebase vêm como códigos em inglês, como auth/invalid-credential ou auth/weak-password. Uma função em src/utils/format.ts converte os principais códigos em mensagens em português, por exemplo "E-mail ou senha incorretos." e "A senha deve ter pelo menos 6 caracteres."

4.3 Modelagem dos dados

Os dados foram organizados em coleções aninhadas dentro do documento de cada usuário, no caminho users/{uid}/listas/{listaId}/produtos/{produtoId}, em que uid é o identificador do usuário no Authentication. Essa estrutura isola as listas de cada pessoa e permite que as regras de segurança comparem o identificador do caminho com o do usuário logado. Cada lista guarda o seu nome e a data de criação. Cada produto guarda os quatro campos da Tabela 1.

A data de adição do produto é gravada com o carimbo de tempo do servidor (serverTimestamp), e não com o relógio do aparelho, para que a ordem dos itens não dependa de a hora do celular estar certa. Como o valor só é preenchido depois que o servidor confirma a gravação, a leitura dos produtos usa a opção serverTimestamps: "estimate", que devolve uma data provisória e evita um campo vazio na tela durante esse intervalo.

A leitura é feita com onSnapshot, função do Firestore que mantém uma conexão aberta e executa uma chamada de retorno a cada mudança nos dados. É esse mecanismo que garante o comportamento em tempo real pedido no enunciado: qualquer alteração na lista chega ao

aplicativo sem que o usuário precise atualizar a tela. Todo o acesso ao banco está reunido no arquivo src/services/listas.ts.

4.4 Regras de segurança

Como o aplicativo conversa diretamente com o Firestore, quem protege os dados são as regras de segurança do banco (FIREBASE, [s. d.]c). A chave de API do Firebase para aplicativos Web não é um segredo, pois qualquer pessoa consegue lê-la no código do aplicativo, e por isso as regras precisam impedir o acesso indevido. O arquivo firestore.rules do repositório permite que um usuário leia, grave e apague somente o que está sob o seu próprio identificador e, ao gravar, confere os campos, como mostra o Quadro 1.

Quadro 1 – Trecho das regras do Firestore para os produtos

```
match /users/{uid}/listas/{listaId} {
  match /produtos/{produtoId} {
    allow read, delete: if request.auth != null
                        && request.auth.uid == uid;
    allow create, update: if request.auth != null
      && request.auth.uid == uid
      && request.resource.data.nomeProduto is string
      && request.resource.data.nomeProduto.size() > 0
      && request.resource.data.quantidade is number
      && request.resource.data.quantidade > 0
      && request.resource.data.precoUnitario is number
      && request.resource.data.precoUnitario >= 0;
  }
}
```

Fonte: elaborado pelo autor (2026).

A mesma validação existe também no aplicativo, antes do envio, para que o usuário receba uma mensagem clara em vez de um erro de permissão. As regras valem como a segunda barreira: mesmo que alguém envie dados por fora do aplicativo, o banco os rejeita.

4.5 Cálculo dos totais

O cálculo está em duas funções simples, apresentadas no Quadro 2. A primeira multiplica a quantidade pelo preço unitário; a segunda soma os totais de todos os produtos. Ambas arredondam o resultado para duas casas decimais, porque a aritmética de ponto flutuante do JavaScript produz resultados como 0,30000000000000004 em somas de valores monetários, o que apareceria na tela.

Quadro 2 – Funções que calculam o total do produto e o total da lista

```
const arredondar = (v: number) => Math.round(v * 100) / 100;

export const totalProduto = (p) =>
  arredondar(p.quantidade * p.precoUnitario);

export const totalLista = (produtos: Produto[]) =>
  arredondar(produtos.reduce(
    (soma, p) => soma + totalProduto(p), 0));
```

Fonte: src/services/listas.ts.

Na tela de produtos, a lista retornada pelo onSnapshot é guardada em um estado do React e o total é recalculado, com useMemo, sempre que essa lista muda. Assim o valor exibido no rodapé acompanha cada inclusão, edição ou exclusão sem nenhuma ação extra do usuário. O formulário também mostra o subtotal do item enquanto ele é digitado, antes de ser salvo. Os campos numéricos aceitam vírgula ou ponto como separador decimal, já que o usuário brasileiro tende a digitar "4,50".

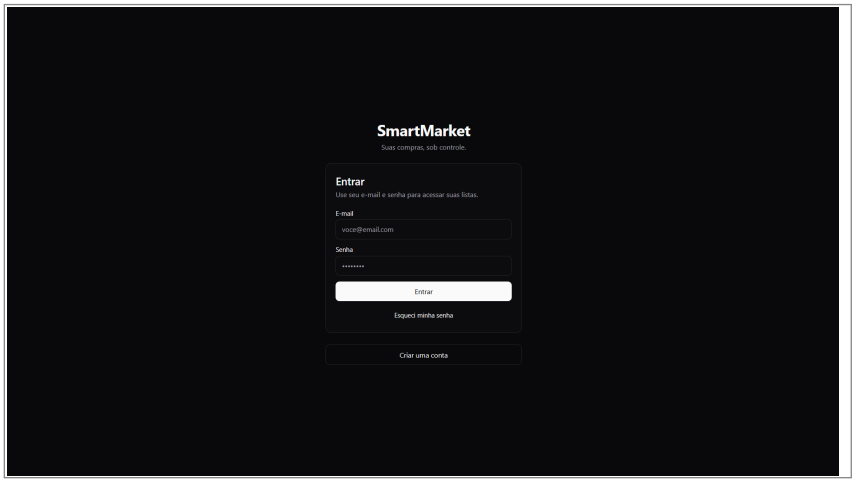
4.6 Interface

A interface usa um tema escuro, com fundo quase preto, bordas finas e botões em tons de cinza, inspirado na aparência dos componentes do shadcn/ui. A biblioteca não foi instalada: os componentes foram escritos à mão, em src/components/ui.tsx, com os elementos nativos do React Native. Foram criados botão (com variantes padrão, contornado, discreto e de exclusão), campo de texto, cartão, rótulo, selo e caixa de aviso. As cores e os raios de borda ficam centralizados em src/theme.ts, o que permite trocar o visual em um único lugar. O tema também foi aplicado ao cabeçalho e ao fundo da navegação, para que não apareçam áreas claras entre as telas.

5 FUNCIONAMENTO DO APLICATIVO

As imagens desta seção foram obtidas com o aplicativo em execução no navegador. A Figura 2 mostra a tela de login. O usuário informa e-mail e senha e entra, ou usa o botão inferior para alternar para o cadastro de uma nova conta. O link "Esqueci minha senha" leva à tela seguinte.

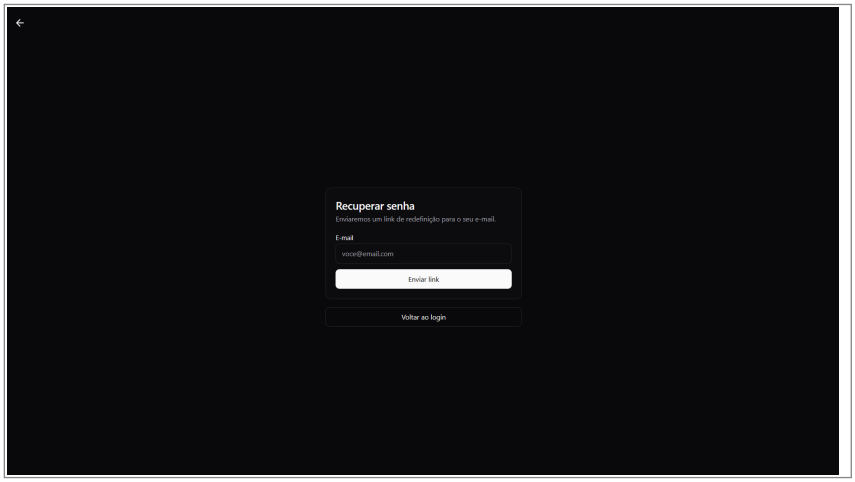
Figura 2 – Tela de login



Fonte: elaborado pelo autor (2026).

Na Figura 3 aparece a tela de recuperação de senha. Basta informar o e-mail cadastrado para que o Firebase envie a mensagem com o link de redefinição. O aplicativo avisa quando o envio é concluído e exibe um texto de erro caso o e-mail seja inválido.

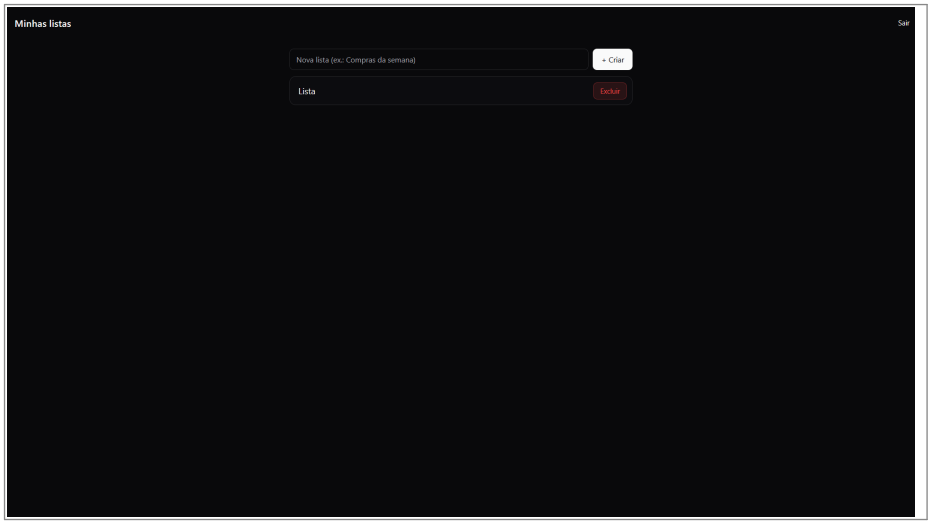
Figura 3 – Tela de recuperação de senha



Fonte: elaborado pelo autor (2026).

Depois do login, o usuário chega à tela de listas, na Figura 4. Ali ele cria uma nova lista digitando o nome e pode excluir as existentes, com confirmação antes da exclusão. Tocar em uma lista abre os seus produtos.

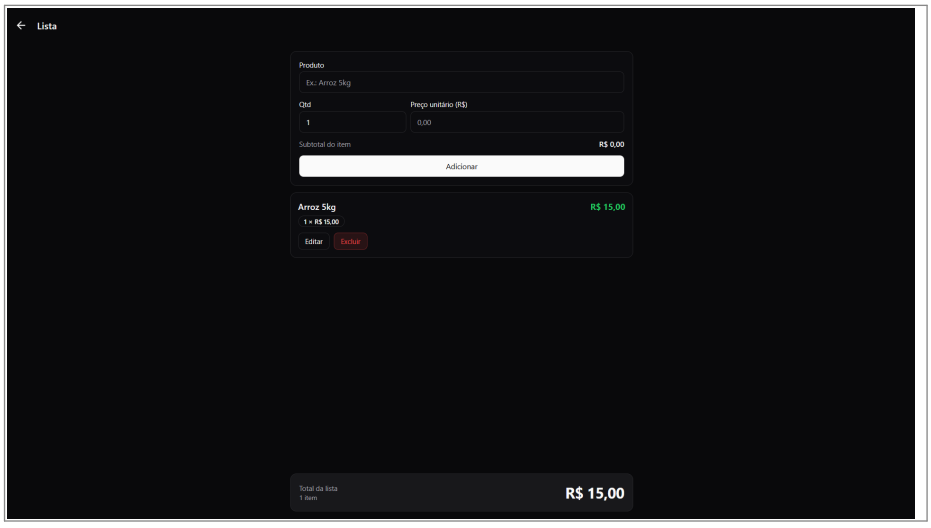
Figura 4 – Tela com as listas de compras do usuário



Fonte: elaborado pelo autor (2026).

A Figura 5 mostra a tela principal do aplicativo. O formulário na parte superior recebe o nome, a quantidade e o preço unitário do produto e mostra o subtotal do item. Cada produto adicionado aparece abaixo, com a quantidade, o preço unitário e o total, além dos botões de editar e excluir. Ao editar, os dados voltam ao formulário e o botão passa a salvar as alterações. O cartão fixo no rodapé exibe o total geral da lista e o número de itens, valores que mudam na hora a cada operação.

Figura 5 – Tela de produtos da lista, com o total geral no rodapé



Fonte: elaborado pelo autor (2026).

6 EXECUÇÃO DO PROJETO

O código está disponível em https://github.com/lucsfn/dam-puc-smartmarket. Para executá-lo é necessário ter o Node.js instalado e um projeto próprio no Firebase, configurado como descrito na seção 4.1. Os passos são os seguintes:

a) clonar o repositório e, na pasta do projeto, instalar as dependências com o comando npm install;
b) copiar o arquivo .env.example para .env e preencher as seis variáveis com os dados do aplicativo Web do projeto no Firebase;
c) no Firebase Console, ativar o login por e-mail e senha e publicar, na aba de regras do Firestore, o conteúdo do arquivo firestore.rules;
d) iniciar o servidor de desenvolvimento com o comando npx expo start -c;
e) abrir o aplicativo no celular, lendo o código exibido no terminal com o Expo Go, ou no navegador, com o comando npm run web.

O parâmetro -c limpa o cache e deve ser usado na primeira execução e sempre que o arquivo .env for alterado, porque o Expo lê as variáveis apenas ao gerar o pacote do aplicativo.

7 CONSIDERAÇÕES FINAIS

O SmartMarket atende ao que foi pedido: tem login e recuperação de senha pelo Firebase Authentication, guarda as listas e os produtos no Cloud Firestore e calcula, em tempo real, o total de cada item e o total da lista. O uso do SDK JavaScript do Firebase simplificou a configuração e permitiu testar o aplicativo tanto no celular quanto no navegador, e as regras de segurança garantem que cada usuário enxergue apenas as próprias listas.

O projeto tem limitações conhecidas. A exclusão de uma lista remove apenas o documento da lista: os produtos dela continuam gravados no banco, embora deixem de aparecer no aplicativo, pois o Firestore não apaga subcoleções automaticamente. Uma melhoria possível é apagar os produtos em lote antes de excluir a lista. Também não foram escritos testes automatizados, e a verificação foi feita de forma manual, percorrendo as telas. Como evolução, o aplicativo poderia permitir marcar os itens já comprados durante a ida ao mercado.

REFERÊNCIAS

EXPO. **Expo documentation**. [S. l.], [s. d.]. Disponível em: https://docs.expo.dev/. Acesso em: 3 nov. 2026.

FIREBASE. **Firebase Authentication**. [S. l.], [s. d.]a. Disponível em: https://firebase.google.com/docs/auth. Acesso em: 3 nov. 2026.

FIREBASE. **Cloud Firestore**. [S. l.], [s. d.]b. Disponível em: https://firebase.google.com/docs/firestore. Acesso em: 3 nov. 2026.

FIREBASE. **Firebase Security Rules**. [S. l.], [s. d.]c. Disponível em: https://firebase.google.com/docs/rules. Acesso em: 3 nov. 2026.

NASCIMENTO, Lucas Fernandes. **SmartMarket**: código-fonte do aplicativo. [S. l.], 2026. Disponível em: https://github.com/lucsfn/dam-puc-smartmarket. Acesso em: 3 nov. 2026.

REACT NAVIGATION. **React Navigation documentation**. [S. l.], [s. d.]. Disponível em: https://reactnavigation.org/docs/getting-started. Acesso em: 3 nov. 2026.